

# HLiquity Emergency Exit: Atomic Batch- Transaction Implementation Guide — Community Edition

How anyone maintaining or mirroring a HLiquity frontend can close the allowance-sweep race window for every user exit flow, without touching the immutable mainnet contracts: HIP-551 atomic batch transactions now, a permissionless redemption router in parallel, and a disciplined wind-down ahead of Hedera's March 2027 removal of smart-contract calls from atomic batches. Includes a complete no-frontend CLI walkthrough and the frontend build, signing-model, UX, security, and test-plan specification. Everything in this document is a proposal that community contributors can adopt, adapt, or take over — nobody mandates anything. **Mainnet-validated (community-2.0):** a community contributor reproduced the batched redemption on mainnet — see §3.3.

---

**Date:** 2026-10-05      **Version:** community-2.0      **Classification:** Public — community security documentation

**Revision:** community-2.0: mainnet validation by a community contributor; corrections to closeTrove approval amount, maxFeePercentage decimals, redemption truncation, router scope, and signing patterns — changelog in the appendix (§9.4).

**Published by the HLiquity community.** No entity operates the HLiquity protocol; this document was produced by community contributors from public mainnet data. Not financial advice. Corrections via GitHub issues only.

**Repository:** <https://github.com/SwisscoastAG/HLiquity> — discussion and corrections via GitHub issues/Discussions.

## 01 TL;DR — executive summary

- **The race window is the vulnerability users can close.** The 2026-09-29 drain executed ~7 seconds after the victim's approval: between the approve transaction and the redeem transaction, the HCHF allowance stood live on-chain and the sweeper bot spent it. **CONFIRMED**  
(community forensic analysis of mainnet data — methodology in §2 and the appendix)
- **No on-chain fix exists.** The HLiquity core contracts are immutable and ownership-renounced — verified on-chain. Every mitigation is off-chain; this document is the engineering guide for that mitigation. **CONFIRMED**
- **The mechanism: HIP-551 atomic batches.** Put `[CryptoApproveAllowance → ContractCall]` into one batch: either every inner transaction commits or all roll back. Mainnet has supported Atomic Batch Transactions since 2025-10-09 (services v0.66). **CONFIRMED**
- **Our shape fits the network rule exactly:** since 2026-09-29 (services v0.77) a batch may contain at most ONE contract operation, and it must be the last inner transaction. Approve-first + redeem-last complies; two contract calls in one batch is not possible. **CONFIRMED**
- **Proven on mainnet by a community contributor (§3.3).** The batched redemption was reproduced on mainnet: inner 1 native `CryptoApproveAllowance` (1 HCHF), inner 2 `TroveManager.redeemCollateral`, one parent AtomicBatch — succeeded, exactly 100,000,000 units consumed and burned, HBAR returned, zero residue allowance; mirror record confirms ordering and shared parent. **Three signing prompts observed.** The assembled wallet flow is upgraded from UNVERIFIED to CONFIRMED (community-validated). **CONFIRMED**
- **Batching removes the inter-transaction gap** in which the bot operates: allowance is set and consumed inside a single consensus transaction, with zero residue if the batch rolls back. This is the only change that closes the 7-second window without touching a contract.  
**RECOMMENDATION** **INFERENCE** (mechanism CONFIRMED; residual-risk claim §04/§08)
- **Hard sunset.** Hedera deprecated smart-contract calls as batch inners on 2026-09-22, with removal planned for “March 2027, estimated”. Batches are a ~5-month bridge, not the endgame. **CONFIRMED** (VERIFIED-NARROWED, §03)
- **Native batches survive the sunset — but that does not help exits.** `CryptoApproveAllowance` + `CryptoTransfer`-style batches remain supported; every HLiquity user exit still needs a contract call, so batches alone cannot carry users past March 2027. **CONFIRMED**
- **Durable fix: a permissionless redemption router** that composes approve+redeem inside one EVM call (user approves the router; the router self-grants wrapper allowance only inside the call and zeroes it after). Survives the sunset. Scope is **redemption only**: `closeTrove/adjustTrove` act on `msg.sender`, so a router cannot perform them for a trove owner — existing-user trove operations ride batches until the cutoff, migration, or v2. `StabilityPool` exits, unstaking, and claims need no router at all (no allowance; standalone calls keep working after March 2027). **RECOMMENDATION**
- **Endgame: v2** with fixed contracts and a trustless merkle-snapshot migration — the essentials of the community migration analysis are summarized in §3.2. Independent of this document.

#### RECOMMENDATION

- **Deadline math:** 2026-10-05 → March 2027 ≈ 20–22 weeks. All batched user exits complete by **2027-01-31**; nothing may depend on contract-call inners past February 2027. **INFERENCE** (from the “estimated” caveat, §03)
- **Frontend path is cheap.** The blokk fork already runs `@hashgraph/hedera-wallet-connect ^1.4.2` + `@hashgraph/sdk 2.72.0` (≥2.64 required) and builds native `ContractExecuteTransactions` — the batch migration is a wrapper change, not a stack change. **Do not touch hashconnect** (deprecated, repo gone). Minimum wallet: HashPack ≥14.4.0. **CONFIRMED**
- **Signing pattern validated on mainnet (§3.3, §6.4):** per-transaction signing — three HashPack prompts (inner 1, inner 2, outer batch) via `hedera-wallet-connect 1.5.1`, then relay of the signed bytes with `hedera_executeTransaction`. The single-prompt `hedera_signAndExecuteTransaction` path remains UNVERIFIED. Issue #694 is a version-dependent caveat, not a blocker for the validated path. **CONFIRMED** (community-validated)

**Open community decisions** — nothing here is mandated; each item is a proposal contributors can adopt, adapt, or take over:

1. **Batch-every-allowance-flow** (7 flows, §06 map) with exact-amount approvals and residue verification (immediate allowance check after every committed batch) — *proposed approach*.
2. **Router contract** — permissionless-redemption scope per the narrowed §3.2 — built in parallel, go/no-go by week 8 (§07) — *looking for a contributor to lead this track*.
3. **Wallet floor:** HashPack ≥14.4.0 as the *recommended minimum*; define the warned non-atomic fallback for older/other wallets (or block and route to the §05 CLI).
4. **Timeline:** *proposed* testnet → mainnet rollout dates per the §07 Gantt, including the 2027-01-31 completion buffer.
5. **V2:** *independent track* per the community migration analysis — it must not block the router.

## 02 Incident recap — what happened and what it cost

All incident facts below come from community forensic analysis of public mainnet data (methodology in this section and the appendix); this section carries only what the build decision depends on. **CONFIRMED**

**Root cause.** CWE-862 (Missing Authorization) confused deputy: the HLiquity token wrappers inherit an ungated `transferFrom` helper (selector `0x15dacbea`) that spends any standing HTS allowance without checking who is asking. Users grant allowances to the *wrapper* contracts (all HTS pulls execute via precompile `0x167`), which is the vulnerable surface. The attacker’s sweeper bot — gas relayer 0.0.995584, receiver 0.0.10393273 — watched for new allowance grants and spent them, typically within seconds. Reference transaction [0.0.995584-1790680920-326001014](#) (view on HashScan): 1,800 HCHF drained from victim 0.0.846778, **7 seconds after the victim’s approval**. The victim’s subsequent protocol redeem executed and reverted atomically (HTS status

292, `SPENDER_DOES_NOT_HAVE_ALLOWANCE` ) — the protocol behaved correctly; the loss happened in the gap between the two user transactions.

2.1 The three critical approvals (the race window lives here)

| # | TOKEN               | USER MUST APPROVE<br>(SPENDER)                  | CONSUMED BY                                                                       |
|---|---------------------|-------------------------------------------------|-----------------------------------------------------------------------------------|
| 1 | HCHF<br>0.0.6070123 | <b>HCHFToken wrapper</b><br><b>0.0.6070122</b>  | provideToSP, repayHCHF, closeTrove, adjustTrove<br>(repay side), redeemCollateral |
| 2 | HLQT<br>0.0.6070128 | <b>HLQTTToken wrapper</b><br><b>0.0.6070127</b> | HLQTStaking.stake                                                                 |
| 3 | LP<br>0.0.6070469   | <b>SaucerSwapPool</b><br><b>0.0.6070133</b>     | SaucerSwapPool.stake                                                              |

Every one of these flows becomes safe when the approval and the consuming call execute atomically — that is the entire design target of this guide. Contract set verified on-chain 2026-10-05 by community fingerprinting (full set in §05 step 3); repo-committed configs are stale and must not be used (§05 step 3, risk §08).

**closeTrove approves NET DEBT, never the full debt (safety-critical).**

`BorrowerOperations.closeTrove` burns **debt – 20 HCHF** from the user and takes the 20 HCHF gas compensation from the GasPool. The approval must therefore be exactly **debt – 20 HCHF**: approving the full debt leaves a standing 20 HCHF allowance to the wrapper — sweepable by the same bot that took the campaign victims. This correction applies everywhere `closeTrove` appears in this guide (§05 step 8, §6.3, §6.6). CONFIRMED (repo-verified)

2.2 Campaign and dollar context

| CAMPAIGN                                                          | HLIQUITY LEG                                                                                                                                                                      | REFERENCE                                                       |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 4 waves, 2026-07-08<br>→ 2026-10-03; 10<br>victims, 5,907.60 HCHF | Jul 8: seven-victim burst in 77 s · Sep 4: 1,750.41<br>HCHF · Sep 29: 1,800 HCHF · Oct 3: 1,780.66 HCHF                                                                           | Community forensics<br>(mainnet data)<br><span>CONFIRMED</span> |
| Full-campaign<br>forensics (all venues)                           | 165 relay calls → 119 committed → 113 distinct<br>debited accounts; plus BankSocial leg: 98 transfers<br>(12.23M BSL + 101.97M sBSL + 6 rUSD via deleted<br>contract 0.0.7444023) | Community forensics<br>(mainnet data)<br><span>CONFIRMED</span> |
| Current open-<br>allowance exposure                               | \$22.18 across 19 owners — small but recurring with<br>every new grant                                                                                                            | Community forensics<br>(mainnet data)<br><span>CONFIRMED</span> |

| MEASURE (US\$)                      | VALUE                                                                                                                                                                                                                                                                   | BASIS                                                                                      |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| HLiquity observed damage            | <b>\$7,451.06</b> (HCHF portion ≈ \$7,113)                                                                                                                                                                                                                              | Community valuation at historical on-chain marks; HCHF ≈ \$1.204 (CoinGecko 2026-10-04)    |
| Same-class losses since 2023        | <b>≈ \$611K–\$636K</b>                                                                                                                                                                                                                                                  | Includes 2023 precompile attack ~\$600K + 2026 campaign venues (community campaign ledger) |
| BankSocial sweep (same crew)        | \$3,036 – \$28.4K (pricing range)                                                                                                                                                                                                                                       | BSL priced; upper bound marks illiquid sBSL at parity                                      |
| WHBAR, measured 16-day window       | ≈ \$790 (7,725.34 WHBAR)                                                                                                                                                                                                                                                | Mirror-node sample, \$0.1022 (CoinGecko 2026-10-04)                                        |
| WHBAR wrapper exposure (not a loss) | <b>\$20.35M supply</b> (199.1M WHBAR, 1,003+ holders)                                                                                                                                                                                                                   | The live surface that dwarfs all realized losses (community wrapper-allowance audit)       |
| Hedera-wide exploit \$ since 2023   | ≈ \$11.6M+ (Sirio ~\$2M Feb 2025; Bonzo ~\$9M Jul 2026)                                                                                                                                                                                                                 | Bonzo attribution caveat: oracle per Bonzo, LayerZero per third parties                    |
| <b>Comparison anchor</b>            | <b>HLiquity is small in \$ next to Bonzo (\$9M) and Sirio (\$2M), but identical in mechanism to the 2023 ~\$600K event and part of the only recurring exploit family on Hedera (~5% of dollars, 2-of-4 major incidents, the only recurring class).</b> <b>CONFIRMED</b> |                                                                                            |

### 03 The external expert assessment, verified

An external engineering assessment of the incident reached community contributors with three candidate mitigations and a warning that the obvious one might be short-lived. Community contributors verified every load-bearing claim against primary sources (HIP, Hedera docs and consensus-node source, the deprecation blog, npm, wallet patch notes, both GitHub forks). Verdict: **the assessment is correct on all three options and on the deprecation, with one narrowing** — the March 2027 date is explicitly an estimate. **CONFIRMED**

“Option 1 — batch transactions: put the approval and the protocol call into a single HIP-551 atomic batch so there is no exposed window. Option 2 — redemption router: a contract that composes approve-and-operate inside one EVM call; durable, but on its own it leaves trove operations behind. Option 3 — v2 redeploy: fixed contracts with a trustless migration, per the community’s migration analysis.”

External engineering assessment received by the community (paraphrase; German original available in the issue tracker). Option numbering is this document’s. The assessor’s “router only redeems” concern stands as a hard scope constraint — a router

cannot carry `closeTrove/adjustTrove` for existing users (they act on `msg.sender`): see the narrowed scope in §3.2 and §08.

### 3.1 Fact-check: the March 2027 deprecation VERIFIED-NARROWED

#### Fact-check box — the deprecation the assessment warned about

Source: official Hedera blog, 2026-09-22 — “Atomic Batch Transactions No Longer Support Smart Contract Calls.” CONFIRMED (the blog exists and says exactly this) · INFERENCE (treating “estimated” as a hard planning date).

“Smart contract calls submitted as inner transactions of an Atomic Batch Transaction enter a six-month deprecation period, with removal planned for March 2027. Atomic batch transactions remain fully supported for composing native Hedera service operations.” Blog quote 1 — deprecation + what survives “As of September 2026: a batch may contain no more than one smart contract call, and it must be the final inner transaction.” Blog quote 2 — current constraint (matches services v0.77 behavior) “From March 2027: smart contract calls are no longer supported as inner transactions of an atomic batch.” Blog quote 3 — the removal “March 2027, estimated.” Blog, timeline — the narrowing: no committed date

**Load-bearing implication.** Our exact mitigation — approve + `ContractCall` in a batch — **stops working around March 2027**. No official March-2027 ban on native batches, atomic swap, or allowance behavior exists (searched; none). Stated rationale for the removal: EVM compatibility — “no equivalent construct in the EVM.” Precedent says Hedera executes such deprecations: the 2023-03-09 precompile attack (~\$600K) was answered first by an emergency proxy-access shutoff and then permanently in services v0.34.5 (blocking `delegatecall`→precompile), followed by the HSCS Security Model v2. Plan for the sunset as real, keep the buffer, and do not let the router depend on batch inners at all.



### 3.2 What this means for the three options

| OPTION                        | VERDICT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | ROLE                                                                |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1 — Atomic batch transactions | Verified against HIP-551, services source, and SDK — <b>and now reproduced on mainnet by a community contributor (§3.3)</b> : works today and closes the window immediately. <b>CONFIRMED</b>                                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Deploy now (weeks)</b> — the stop-gap bridge until ~March 2027   |
| 2 — Redemption router         | Sound and durable past the sunset (composition inside the EVM is Hedera's own recommended replacement). <b>Scope narrowed (community-2.0)</b> : a durable router can carry <b>permissionless redemption only</b> — closeTrove/adjustTrove act on <code>msg.sender</code> , so a router would be the caller, not the trove owner, and cannot perform those operations for existing users. SP exits, unstaking, and claims need <b>no router</b> (no allowance involved; standalone calls keep working after March 2027). Existing-user trove operations = batches until cutoff / migration / v2. <b>RECOMMENDATION</b> | <b>Build in parallel (weeks 3–16)</b> — the durable redemption path |
| 3 — v2 redeploy               | Correct endgame; a separate community migration analysis exists, and the essentials are summarized here: immutable contracts → all mitigations off-chain → v2 with trustless merkle-snapshot migration is the endgame. <b>RECOMMENDATION</b>                                                                                                                                                                                                                                                                                                                                                                          | <b>Decision track, weeks 5–20</b> — independent of this guide       |

**Why “native batches stay supported” matters — and why it does not save the exits.** After the sunset, CryptoApproveAllowance + CryptoTransfer-style atomic batches remain fully supported. But no allowance-carrying HLiquidity exit can be expressed without a contract call: redeeming HCHF, closing a trove, repaying, adjusting, staking — all are `ContractCall`s. The surviving native capability is therefore irrelevant to those exits, and the batch is correctly sized as a bridge, with the router as the durable crossing *for redemption*. Operations with no allowance — StabilityPool withdrawals, unstaking, claims, `claimCollateral` — keep working as ordinary standalone contract calls after March 2027 (no batch, no router needed). The allowance-carrying trove operations must complete inside the batch window — or, for redemption, move to the router. **CONFIRMED**

### 3.3 Community mainnet validation — PROVEN

#### Community mainnet validation — proven **PROVEN**

A community contributor reproduced the batched redemption **end-to-end on mainnet** and reported it in the GitHub issue (contributor unnamed unless they request attribution; test evidence in the issue). **CONFIRMED** (community-validated)

- **Reproduction:** inner 1 = native `CryptoApproveAllowance` (1 HCHF = 100,000,000 units, 8-dec) to the HCHF wrapper; inner 2 = `TroveManager.redeemCollateral` as the final inner — both under one parent AtomicBatch. Exactly the shape this guide specifies, satisfying the one-contract-op-last rule in practice.

- **Result:** the batch succeeded; **exactly 100,000,000 units consumed and burned**; HBAR collateral returned to the user; **residue allowance zero**.
- **Mirror record** confirms the inner ordering and the shared parent (type `ATOMICBATCH` ; inners linked by `parent_consensus_timestamp` / `nonce` ).
- **Signing observed: three signing prompts** — inner 1, inner 2, outer batch — i.e. per-transaction signing via hedera-wallet-connect 1.5.1, with the signed bytes relayed via `hedera_executeTransaction` (details in §6.4 and the issue).

**Status upgrades (UNVERIFIED → CONFIRMED, community-validated):** the assembled hedera-wallet-connect + BatchTransaction flow; the no-operator freeze + wallet-signing pattern (§6.2); HashPack ≥14.4.0 batch signing in practice (via connector 1.5.1).

**Still UNVERIFIED:** the single-prompt `hedera_signAndExecuteTransaction` path for a batch containing a contract call; how HashPack renders the inner details inside each signing prompt.

### 3.4 Community guidance — what members should tell each other

This section is written to be quoted directly — in Discord, Telegram, X, or a README banner. Short version first, then the details behind it.

#### Member fact-sheet — quote freelyShareable

- **What happened.** A sweeper bot has been draining standing token allowances on Hedera — including HLIquity's HCHF — typically within seconds of a user's approval. The HLIquity leg: 10 victims, 5,907.60 HCHF, four waves from 2026-07-08 to 2026-10-03. The protocol contracts behaved correctly; the loss happens in the seconds between a user's approve and the user's redeem. **CONFIRMED**
- **Who is at risk.** Anyone holding an open (unspent) HCHF, HLQT, or LP allowance to the wrapper contracts — the allowance stays sweepable until it is spent or revoked — and anyone who creates a fresh non-atomic approval.
- **What to do now.** (1) Do not create fresh non-atomic HCHF/HLQT/LP approvals. (2) Check your open allowances and revoke unneeded ones on HashScan (via the mirror-node endpoint below). (3) If you need to exit — redeem HCHF, close a trove, leave the StabilityPool, unstake — use the batched CLI from this repository (§05) and test with a small amount first; for closeTrove, approve exactly debt – 20 HCHF, never the full debt (§2.1). (4) Verify every contract and token address against the verified set in §05 step 3 — counterfeit HLQT exists. **RECOMMENDATION**
- **What never to do.** Never approve "unlimited" amounts. And: nobody will ever DM you a refund — anyone offering compensation is a scammer. Always. **CONFIRMED** (as community policy)

**Scam warning — expect refund-impersonation waves.** Publishing this document will likely trigger DMs from accounts impersonating "the HLIquity team", a "HLiquity foundation", or a "compensation desk". The community never DMs first, has no treasury, and sends no compensation. There is no refund flow, no compensation program, and no official support account. Treat every unsolicited refund offer as a scam; report and block, and warn others using the fact-sheet above.



### How to contribute

- **Review this document** — corrections, however small, via GitHub issues.
- **Run it** — execute the §05 CLI scenarios and the §6.7 test plan on testnet, and report what you find (scenario T2: three prompts are now confirmed — the remaining question is what HashPack renders *inside* each prompt).
- **Pick up a track** — the frontend work (§06) and the redemption-router track (§3.2, §08) both need owners.
- **Improve the IOC list** — new sweeper addresses, watcher rules, flagging evidence.

No permission needed — that's the point.

### How to verify

Every address, transaction, and claim in this document is checkable against public infrastructure. Two endpoints cover almost everything:

- **Mirror node REST:** `https://mainnet-public.mirrornode.hedera.com/api/v1` — transactions, allowances, token relationships (§9.2 query list).
- **HashScan:** `https://hashscan.io/mainnet` — the reference drain transaction is at `/transaction/0.0.995584-1790680920-326001014`.

Don't trust — verify, then open an issue if something doesn't match.

## 04 Why atomic batches close the race

The sweeper bot wins because the vulnerable state — a live, unspent allowance to the wrapper — exists *between* two user transactions, visible to anyone polling the mirror node (bot cycle ~30–60 s, response in seconds — community monitoring of mainnet data, §2). HIP-551 inner transactions execute sequentially, in listed order, inside a single consensus transaction. In a batch, the allowance transitions *nonexistent* → *set* → *consumed* → *zero* with no inter-transaction gap, and if the redeem fails, the approve rolls back with it — no residue for the bot to take later. **CONFIRMED** (semantics, network source) **INFERENCE** (race-elimination claim — strongest available, no window exists by construction; see §08 residual risks) One honest caveat, added in community-2.0: a *committed* redeem can consume **less** than approved when the redemption hints truncate the amount — approve the truncated amount and verify the residue allowance immediately after (§05 steps 4d–5).

TODAY — two separate transactionsexposed by construction

tx 1user signs `approve` — HCHF → wrapper 0.0.6070122, exact amount

approval confirms — allowance is now live and sweepable

**EXPOSED WINDOW (7 s in the reference case).** Sweeper bot 0.0.995584 spends the allowance via the wrapper's ungated `transferFrom` ( `0x15dacbea` ) — a plain HTS transfer, not a protocol interaction.

tx 2user's `redeemCollateral` executes — HCHF already gone; redeem reverts atomically (status 292). Loss already settled in tx 1's gap.

→ to retry, the user must re-approve — recreating the exact state the bot is watching for.

BATCHED — one atomic unit (HIP-551) no window exists — proven on mainnet (§3.3)

inner 1 `CryptoApproveAllowance` — HCHF → wrapper, exact amount (native op, any position)

inner 2 `ContractCall` — `TroveManager.redeemCollateral` (the ONE contract op; last by rule)

executes in listed order within a single consensus transaction — no inter-transaction gap

both commit — or both roll back ( `INNER_TRANSACTION_FAILED` ); residue = 0 on rollback; on commit, residue = 0 whenever the call consumes the exact approved amount (fixed-amount flows;

redemption: approve the truncated amount — §05 step 4d)

#### 4.1 The one-contract-op-last rule — and why our shape fits

Mainnet constraint since 2026-09-29 (services v0.77, `AtomicBatchHandler.pureChecks`): a batch may contain at most ONE inner transaction that is a contract operation ( `CONTRACT_CALL` , `CONTRACT_CREATE_INSTANCE` , or `ETHEREUMTRANSACTION` ), and it must be the last inner; anything else pre-checks `INVALID_TRANSACTION_BODY` . Forbidden inner types (network blacklist): Freeze, AtomicBatch (no nesting), ScheduleCreate, ScheduleSign. Allowed native types include `CRYPTOAPPROVEALLOWANCE` and `CRYPTOTRANSFER` . Our mitigation shape `[CryptoApproveAllowance, ContractCall]` satisfies the rule exactly — confirmed in practice by the community mainnet reproduction (§3.3). Two contract calls in one batch (e.g. approve-token + redeem as two EVM calls) is **not possible** — approve must be the native HTS allowance transaction. **CONFIRMED**

#### 4.2 Batch limits and cost model **CONFIRMED**

| PROPERTY           | VALUE                                                                                                                                                            | ENGINEERING CONSEQUENCE                                                                                                                                                             |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inner count        | ≤ 50 (network config; SDK JSDoc stale-says 25)                                                                                                                   | Plan ≤ 25 to be safe; our flows use 2 (3 with association)                                                                                                                          |
| Total size         | ≤ 6 KB per batch                                                                                                                                                 | 7-arg <code>redeemCollateral</code> + approve fits comfortably                                                                                                                      |
| Valid duration     | standard ~180 s                                                                                                                                                  | User must sign promptly; stale batches expire unspent                                                                                                                               |
| Inner node account | forced to 0.0.0; each inner has own payer + transactionId; dedup individually                                                                                    | Set explicit inner transactionIds when the inner payer differs from the client operator; re-submitting needs fresh ids                                                              |
| Fees               | outer payer pays batch node+network fee; <b>each inner pays its own full fees even if the batch later rolls back</b> (inners before the failing one are charged) | A failed redeem costs approve-side fees too — show honest max-fee estimates; budget 2× inner fees + batch fee                                                                       |
| Signing            | every inner needs <code>batchKey</code> set (public key); the outer must be signed by ALL distinct <code>batchKey</code> private keys + the batch payer          | If the user's key is <code>batchKey</code> on every inner, that single key signs everything — the model wallets already implement (three prompts observed in the §3.3 mainnet test) |

## 05 Tutorial: batch transactions without a frontend (CLI)

Every flow in this guide can be executed from a Node.js CLI — no frontend, no wallet extension, using only SDK APIs verified in §01/§03. A complete reference implementation of this tutorial (plus the §06 frontend pattern in TypeScript) is published alongside this document in the repository under `code/` ( `code/cli`, `code/frontend` ). **Testnet first, then mainnet with small amounts.** RECOMMENDATION

### Step 1 — Install

Node 18+. The BatchTransaction API requires `@hashgraph/sdk`  $\geq 2.64.0$  (also shipped as `@hiero-ledger/sdk` 2.89.x). CONFIRMED

```
# package.json (excerpt)
{
  "type": "commonjs",
  "dependencies": {
    "@hashgraph/sdk": "^2.72.0",
    "dotenv": "^16.0.0"
  }
}

# install
npm install
```

### Step 2 — Environment and key handling

The CLI runs with a payer account — in SDK terms the client *operator* — that pays every fee: the batch fee and each inner's full fee, including inner fees already spent when a batch later rolls back (§4.2). Keys come from the environment or a permission-locked file; never hardcode them, never commit them, never put them anywhere near frontend code. CONFIRMED (fee model) /

RECOMMENDATION (handling)

```
# .env - chmod 600 .env; never commit
# TESTNET FIRST: use a testnet account and Client.forTestnet() until the
# flow is fully validated; move to mainnet only for small-amount pilots.
OPERATOR_ID=0.0.YOURACCOUNT
OPERATOR_KEY=302e020100300506032b657004220420...

require("dotenv").config();
const { Client, PrivateKey, AccountId } = require("@hashgraph/sdk");

const operatorKey = PrivateKey.fromString(process.env.OPERATOR_KEY);
const client = Client.forMainnet() // Client.forTestnet() during validation
  .setOperator(AccountId.fromString(process.env.OPERATOR_ID), operatorKey);
```

### Step 3 — Pin the verified contract set (do not trust repo configs)

The mainnet deployment was never committed to the repo; the committed `hedera.json` is testnet-era and mislabeled. The set below was fingerprinted on mainnet on 2026-10-05. **CONFIRMED**

| NAME                        | CONTRACT ID               | ROLE IN THE BATCHED FLOWS                                                                           |
|-----------------------------|---------------------------|-----------------------------------------------------------------------------------------------------|
| BorrowerOperations          | 0.0.6070094               | openTrove, addColl, withdrawColl, withdrawHCHF, repayHCHF, closeTrove, adjustTrove, claimCollateral |
| TroveManager                | 0.0.6070118               | redeemCollateral (batched); liquidations (no batch)                                                 |
| StabilityPool               | 0.0.6070108               | provideToSP (batched); withdrawFromSP / withdrawETHGainToTrove (no batch)                           |
| HLQTStaking                 | 0.0.6070064               | stake (batched); unstake (no batch)                                                                 |
| SaucerSwapPool (LP rewards) | 0.0.6070133               | LP stake (batched); withdraw / claimReward / withdrawAndClaim (no batch)                            |
| HCHF wrapper / token        | 0.0.6070122 / 0.0.6070123 | allowance spender for every HCHF-consuming flow                                                     |
| HLQT wrapper / token        | 0.0.6070127 / 0.0.6070128 | allowance spender for HLQT staking                                                                  |
| LP token (uniToken)         | 0.0.6070469               | allowance token for LP staking                                                                      |
| HintHelpers / PriceFeed     | 0.0.6070047 / 0.0.6070082 | redeem hint derivation + price for the CLI pre-flight                                               |

### Step 4 — Pre-flight: association check, price, and redemption hints

`redeemCollateral` needs a price and three hints. Both come from read-only queries — free, and they catch the failure modes before any fee is spent. If the account is not yet associated to HCHF, add a `TokenAssociateTransaction` as the batch's first inner (association is a native op, allowed in any position). **CONFIRMED**

```
const {
  ContractCallQuery, ContractFunctionParameters, AccountInfoQuery
} = require("@hashgraph/sdk");

const HCHF = require("@hashgraph/sdk").TokenId.fromString("0.0.6070123");
const PRICE_FEED = "0.0.6070082", HINT_HELPERS = "0.0.6070047";

// 4a) association check (before spending an inner tx on it)
const info = await new AccountInfoQuery()
  .setAccountId(client.operatorAccountId).execute(client);
const associated = info.tokenRelationships.get(HCHF.toString()) !== undefined;
```

```

console.log("HCHF associated:", associated);

// 4b) PriceFeed.lastGoodPrice() – public getter
const price = await new ContractCallQuery()
  .setContractId(PRICE_FEED).setGas(200_000)
  .setFunction("lastGoodPrice").execute(client);
const lastGoodPrice = price.getUint256(0);

// 4c) HintHelpers.getRedemptionHints(amount, price, maxIterations) – maxIterations is a
// deliberate cap: higher scans more troves per call at higher gas. Choose it on purpose.
const hints = await new ContractCallQuery()
  .setContractId(HINT_HELPERS).setGas(1_200_000)
  .setFunction("getRedemptionHints",
    new ContractFunctionParameters()
      .addUint256(amount) // HCHF amount, 8-dec raw units
      .addUint256(lastGoodPrice)
      .addUint256(maxIterations))
  .execute(client);
// decode per ABI: (address firstHint, uint partialNICR, uint truncatedHCHFAmount)
const firstHint = hints.getAddress(0);
const partialNICR = hints.getUint256(1);
const truncatedAmount = hints.getUint256(2); // what the call can ACTUALLY consume

// 4d) TRUNCATION RULE – never approve more than the call will consume:
// if truncatedAmount < requested amount, EITHER abort and re-run this pre-flight
// with fresh hints / a higher maxIterations, OR set amount = truncatedAmount and
// continue. Even then, stale hints or a state change between pre-flight and
// execution can leave partial consumption – check the residue allowance
// immediately after execution (step 7).

```

## Step 5 — Build the two innerers (approve first, contract call last)

Approve the amount the call will **actually consume** — for redemption, the **truncated amount from step 4d**, never more. Zero residue is guaranteed only when the call consumes the exact approved amount: true by construction for the fixed-amount flows (provideToSP, repayHCHF, stake), **not** guaranteed for redemption (truncated or stale hints can leave partial consumption) — hence the mandatory residue check in step 7. `batchify(client, key)` sets the batch key, signs with the payer key, and freezes (forcing node account 0.0.0). Hint addresses are EVM long-zero addresses — convert contract IDs with `AccountId.toEvmAddress()`. **CONFIRMED**

```

const {
  AccountAllowanceApproveTransaction, ContractExecuteTransaction,
  TransactionId, AccountId
} = require("@hashgraph/sdk");

```

```

const TROVE_MANAGER = "0.0.6070118";
const HCHF_WRAPPER = AccountId.fromString("0.0.6070122"); // the spender
const ZERO_ADDR = AccountId.fromString("0.0.0").toEvmAddress(); // TEST DEFAULT only – see 4c/4d

// inner 1 – native allowance tx (NOT a contract call).
// `amount` here = the truncated amount from step 4d – never more than the call consumes.
const approveTx = new AccountAllowanceApproveTransaction()
    .approveTokenAllowance(HCHF, client.operatorAccountId, HCHF_WRAPPER, amount)
    .setTransactionId(TransactionId.generate(client.operatorAccountId));
await approveTx.batchify(client, operatorKey);

// maxFeePercentage – 8-DECIMAL fixed point on THIS deployment (BaseMath DECIMAL_PRECISION = 1e8)
// 100% = 1e8; the 0.5% borrow-fee floor = 500_000. Do not port percent constants from
// Ethereum-forked codebases – the decimal scale here is different.
const maxFeePercentage = 100_000_000; // = 100% – set your cap deliberately

// inner 2 – the SINGLE contract op; must be LAST. Gas 3,000,000 is a TEST DEFAULT –
// calibrate on testnet before mainnet use.
const redeemTx = new ContractExecuteTransaction()
    .setContractId(TROVE_MANAGER)
    .setGas(3_000_000)
    .setFunction("redeemCollateral",
        new ContractFunctionParameters()
            .addUint256(amount) // = truncatedAmount from step 4d
            .addAddress(firstHint) // or ZERO_ADDR with the test-default fallback
            .addAddress(ZERO_ADDR) // upperPartialRedemptionHint
            .addAddress(ZERO_ADDR) // lowerPartialRedemptionHint
            .addUint256(partialNICR)
            .addUint256(maxIterations) // capped deliberately, e.g. per gas calibration
            .addUint256(maxFeePercentage)) // 8-dec percent: 100% = 1e8, 0.5% floor = 500_000
    .setTransactionId(TransactionId.generate(client.operatorAccountId));
await redeemTx.batchify(client, operatorKey);

```

## Step 6 — Assemble, sign, execute

`addInnerTransaction` throws unless the inner is frozen with a `batchKey` set (that is what `batchify` did). The outer must then be signed by every distinct `batchKey` key plus the payer — with one key as `batchKey` on all inner, that key signs everything. **CONFIRMED**

```

const { BatchTransaction } = require("@hashgraph/sdk");

const batch = new BatchTransaction()
    .addInnerTransaction(approveTx) // first
    .addInnerTransaction(redeemTx); // last – the one contract op
batch.setTransactionId(TransactionId.generate(client.operatorAccountId));

```



```
await batch.freezeWith(client);
await batch.sign(operatorKey); // all batchKey keys + payer

const response = await batch.execute(client);
const receipt = await response.getReceipt(client);
console.log("batch status:", receipt.status.toString());
console.log("batch id:", batch.transactionId.toString());
```

## Step 7 — Receipts and mirror-node verification

The JS SDK exposes **property getters** ( `batch.innerTransactions` , `batch.innerTransactionIds` ) — the Java-style `getInnerTransactionReceipts()` does not exist in JS. Query each inner receipt by id, then confirm on the mirror node: the parent is type `ATOMICBATCH` , inners carry `batch_key` , `parent_consensus_timestamp` , and `nonce` ; and the owner's residue allowance must be zero — **check it immediately after the batch commits**, because a redemption that consumed less than approved (truncation or a stale-hint race) leaves residue that must be revoked at once.

CONFIRMED

```
const { TransactionReceiptQuery } = require("@hashgraph/sdk");
for (const id of batch.innerTransactionIds) { // property getters, not methods
  const r = await new TransactionReceiptQuery()
    .setTransactionId(id).execute(client);
  console.log(id.toString(), "->", r.status.toString());
}
```

base: <https://mainnet-public.mirrornode.hedera.com/api/v1> # testnet:

testnet.mirrornode.hedera.com # parent (ATOMICBATCH) + all inners, linked by

parent\_consensus\_timestamp / nonce GET /transactions/{batchId} # residue check: the approved amount must be fully consumed (allowance value 0) GET /accounts/{owner}/allowances/tokens

## Step 8 — The generic pattern for all 7 batched flows

Identical skeleton for every allowance-consuming flow — only the token, spender, target contract, function, and gas change:

- 1. inner 1 (native):** `AccountAllowanceApproveTransaction` — exact amount, correct spender (wrapper 0.0.6070122 for HCHF, wrapper 0.0.6070127 for HLQT, SaucerSwapPool 0.0.6070133 for LP). Optionally prepend `TokenAssociateTransaction` on first use. For **closeTrove the exact amount is net debt (debt – 20 HCHF), never the full debt** — see the table note below.
- 2. inner 2 (the ONE contract op, last):** the flow's `ContractExecuteTransaction` .
- 3.** Assemble → freeze → sign (batchKey keys + payer) → execute → per-inner receipts → mirror-node residue check **immediately after commit**.

| FLOW                                                                                                                                                                                                                                                | BATCH CONTENTS                                                                                                                                                      | GAS — TEST<br>DEFAULTS<br>(CALIBRATE ON<br>TESTNET) |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| redeemCollateral<br><b>the incident flow</b>                                                                                                                                                                                                        | [approve HCHF→wrapper exact] + [redeemCollateral]<br>— hints via HintHelpers (§05 step 4); approve and call<br>with the <b>truncated</b> amount (step 4d), or abort | 3,000,000                                           |
| provideToSP                                                                                                                                                                                                                                         | [approve HCHF→wrapper exact] + [provideToSP]                                                                                                                        | 800,000                                             |
| repayHCHF                                                                                                                                                                                                                                           | [approve exact] + [repayHCHF]                                                                                                                                       | 1,500,000                                           |
| closeTrove                                                                                                                                                                                                                                          | [approve exact <b>net debt = debt – 20 HCHF</b> ] +<br>[closeTrove]                                                                                                 | 1,500,000                                           |
| adjustTrove (repay or<br>withdraw coll)                                                                                                                                                                                                             | [approve exact] + [adjustTrove] (+ payable when<br>adding collateral)                                                                                               | 1,500,000                                           |
| stake HLQT                                                                                                                                                                                                                                          | [approve HLQT→HLQT wrapper exact] + [stake]                                                                                                                         | 500,000                                             |
| stake LP                                                                                                                                                                                                                                            | [approve LP→SaucerSwapPool exact] + [stake]                                                                                                                         | 500,000                                             |
| <b>No batch (no allowance):</b> openTrove / addColl / withdrawColl / withdrawHCHF (single ContractExecute;<br>set payable amount in HBAR for openTrove/addColl) · unstake / withdraw / claims · liquidations ·<br>claimCollateral. <b>CONFIRMED</b> |                                                                                                                                                                     |                                                     |

**closeTrove safety note.** `BorrowerOperations.closeTrove` burns debt – 20 HCHF from the user and takes the 20 HCHF gas compensation from the GasPool. Approve exactly the net debt: approving the full debt leaves a standing 20 HCHF allowance to the wrapper — stealable by the same sweep mechanism this guide exists to close (§2.1).

**CONFIRMED** (repo-verified)

**Test defaults, not guarantees.** The gas figures above and the address(0)-hints fallback in steps 4c/4d are **TEST DEFAULTS** — calibrate gas on testnet, use live hints from HintHelpers in production, and cap `maxIterations` deliberately (it bounds how many troves one call scans).

## Step 9 — Troubleshooting

| SYMPTOM                                              | MEANING                                                                                         | FIX                                                                                                                                                                                     |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>INNER_TRANSACTION_FAILED</code>                | First non-SUCCESS inner aborted the whole batch; inners before it are still charged their fees. | Read per-inner receipts / mirror node to find which inner and why. For redeem: re-run the §05 step-4 pre-flight (balance, hints, price freshness). Then rebuild — do not blindly retry. |
| <code>INVALID_TRANSACTION_BODY</code><br>(pre-check) | Constraint violated: more than one contract op, or the contract op is not the last inner.       | Exactly one <code>ContractExecuteTransaction</code> , positioned last; the approve must be the native <code>AccountAllowanceApproveTransaction</code> .                                 |
| <code>BUSY</code>                                    | Node busy at submission.                                                                        | Wait, rebuild with fresh inner <code>transactionIds</code> , resubmit. (Inners dedup individually.)                                                                                     |
| <code>PAYER_SIGNATURE_CHECK_FAILED</code>            | Outer batch not signed by all distinct batchKey private keys + the payer.                       | With one batchKey on all inners: sign the outer with that same key (plus the payer signature).                                                                                          |
| SDK throws at <code>addInnerTransaction</code>       | Inner not frozen or batchKey missing.                                                           | Call <code>batchify(client, key)</code> (or manual <code>setBatchKey</code> + <code>freezeWith</code> + <code>sign</code> ) before adding.                                              |
| Receipts query returns nothing for an inner id       | Wrong id form or batch not yet processed.                                                       | Use <code>batch.innerTransactionIds</code> verbatim; poll with backoff; mirror node <code>GET /transactions/{batchId}</code> as ground truth.                                           |

## 06 Frontend recommendation — React + HashPack

### 6.1 Stack facts

**DO NOT build on hashconnect — it is dead.** Final release 3.0.14 (2025-10-02); the npm package carries the banner “shut down by 2026, upgrade to WalletConnect”; the GitHub repo is now 404. HashPack 14.3.0 (2026-06-24) removed the legacy HashConnect v2 protocol. Any hashconnect dependency is a liability and must be removed, not merely avoided. **CONFIRMED**

- **HashPack ≥ 14.4.0 (2026-07-17) supports batch transactions over WalletConnect** — “Dapps can now bundle an associate + transfer in a single approval.” 14.4.1 added dapp-auto-approval and removed the flaky WalletConnect auto-associate that “didn’t work well with batch

transactions.” This is the minimum wallet version for the batched UX — and its batch signing is now **validated in practice** by the community mainnet test (§3.3). **CONFIRMED**

- **Connector:** `@hashgraph/hedera-wallet-connect` (**HIP-820**) — latest 2.1.x line in 2026. HIP-820 methods: `hedera_signAndExecuteTransaction` (base64 transactionList + signerAccountId; wallet signs *and* submits), `hedera_signTransaction` (sign-only), `hedera_executeTransaction` (relay pre-signed). The validated signing flow uses sign-only per transaction + relay (§6.4).

**CONFIRMED**

- **The blokk fork already qualifies.** `blokk-studio/Hliquity` runs `@hashgraph/hedera-wallet-connect ^1.4.2` + `@hashgraph/sdk 2.72.0` (≥2.64 required — satisfied) and builds a native `ContractExecuteTransaction` per call today. **The batch migration is a wrapper change, not a stack change.** **CONFIRMED** (evaluating the 2.1.x connector line: §08 open questions)

- **What remains unverified:** how HashPack renders the inner details inside each signing prompt (no public screenshots/descriptions; the mainnet test observed *three prompts* but did not settle the rendering question), and the single-prompt `hedera_signAndExecuteTransaction` path for a batch containing a contract call. Both are test-plan items (§6.7), not assumptions. **UNVERIFIED**

(the assembled hedera-wallet-connect + BatchTransaction flow itself is now community-validated — §3.3)

## 6.2 Architecture: BatchTxBuilder around the existing builders

Keep every existing `ContractExecuteTransaction` builder untouched. Add one `BatchTxBuilder` that wraps a flow: build approve inner (native, exact amount, correct spender) → take the flow’s existing contract-execute builder as the final inner → set explicit inner transactionIds (`TransactionId.generate(userAccount)`) → set the user’s public key as batchKey on every inner → freeze for the wallet signer (no SDK operator client exists in the frontend — the wallet is the signer). This no-operator freeze + wallet-signing pattern was assembled for the previous community edition and is **now validated on mainnet** via connector 1.5.1 (§3.3) — keep it behind the §6.7 regression tests on every connector bump. Batch assembly and the one-contract-op-last invariant are enforced in one place, so the 7 flows cannot drift out of compliance. Reference TypeScript (BatchTxBuilder, the 7 flow builders, the React hook, and the HashPack version gate) is published alongside this document in the repository under `code/frontend/`.

### 6.3 Flow → batch map (the frontend contract)

| FLOW                                        | BATCH CONTENTS                                                                | NOTES                                                                                                                    |
|---------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| openTrove                                   | <b>NO BATCH</b> — single ContractExecute, <code>setPayableAmount(HBAR)</code> | no allowance involved                                                                                                    |
| addColl /<br>withdrawColl /<br>withdrawHCHF | <b>NO BATCH</b>                                                               | no allowance                                                                                                             |
| redeemCollateral                            | [Approve HCHF→wrapper 0.0.6070122 exact] + [ContractExecute redeemCollateral] | <b>the incident flow</b> ; hints via HintHelpers 0.0.6070047; approve the <b>truncated</b> amount or abort (§05 step 4d) |
| provideToSP                                 | [Approve HCHF→wrapper exact] + [provideToSP]                                  |                                                                                                                          |
| repayHCHF                                   | [Approve exact] + [repayHCHF]                                                 |                                                                                                                          |
| closeTrove                                  | [Approve exact <b>net debt = debt – 20 HCHF</b> ] + [closeTrove]              | GasPool supplies the 20 HCHF compensation (§2.1) — approving the full debt leaves 20 HCHF sweepable                      |
| adjustTrove (repay or withdraw coll)        | [Approve exact] + [adjustTrove] (+ payable if adding coll)                    | allowance only on the repay side                                                                                         |
| stake HLQT                                  | [Approve HLQT→wrapper 0.0.6070127 exact] + [stake]                            |                                                                                                                          |
| stake LP                                    | [Approve LP→SaucerSwapPool 0.0.6070133 exact] + [stake]                       |                                                                                                                          |
| unstake / withdraw / claims                 | <b>NO BATCH</b>                                                               | no allowance; standalone calls keep working after March 2027                                                             |
| first-time token use                        | [TokenAssociate(HCHF/HLQT/LP)] as native inner <i>before</i> the approve      | association is not a contract op                                                                                         |

### 6.4 Signing model — validated pattern, plus the #694 version caveat

**Validated on mainnet (§3.3): per-transaction signing, three prompts.** The dapp builds and freezes the batch without an operator, then the wallet signs **each inner transaction individually and the outer batch separately — three signing prompts** — using the HIP-820 sign method of hedera-wallet-connect 1.5.1; the dapp then relays the signed bytes with `hedera_executeTransaction`. This validates the no-operator freeze + wallet-signing pattern of §6.2 end-to-end, with HashPack ≥14.4.0 as the signing wallet. **CONFIRMED** (community-validated, connector 1.5.1)

**Version-dependent caveat — hedera-wallet-connect issue #694.** On some connector versions, `DAppSigner.signTransaction` rebuilds the `ContractExecute` body and returns a signature over the rebuilt bytes, not the original — which breaks naive sign-then-submit for a batch containing a contract call. The validated 1.5.1 per-transaction path signs the original bytes and is **not** affected. Present #694 as a re-test trigger on connector bumps — not as a blanket mandate against signing-and-relaying, and not as a reason to prefer the unvalidated single-prompt path. **CONFIRMED** (issue open) / **INFERENCE** (scope by version)

**Separate path, still unvalidated:** single-prompt `hedera_signAndExecuteTransaction` (wallet signs and submits in one prompt) for a batch containing a contract call. No community test has validated it; keep it behind a testnet gate until scenario T3/T2-style evidence exists. Wire format for both paths: base64 `transactionList` + `signerAccountId`. After relay/submission, poll the mirror node for the batch id and inner statuses (parent `ATOMICBATCH`, inners linked by `parent_consensus_timestamp / nonce`) and only then resolve the UI promise — the wallet's return is the submission, not finality. **UNVERIFIED** (single-prompt path) / **CONFIRMED** (wire format; mirror-node model)

## 6.5 UX requirements

- **Anticipate three signing prompts — and explain them before the wallet opens.** The validated wallet flow (§6.4) presents one prompt per inner transaction plus one for the outer batch (inner 1, inner 2, outer). A user expecting a single prompt will suspect a bug or an attack: the UI must say three prompts are coming and show a plain-language line for each inner (approve X HCHF to wrapper 0.0.6070122; redeem X HCHF for collateral). What HashPack renders *inside* each prompt is **UNVERIFIED** — until scenario T2 lands, treat wallet rendering as unknown and make the dapp-side preview authoritative.
- **Exact-amount display, always.** Show the precise approve amount in HCHF/HLQT/LP units; never “unlimited”, never rounded. For closeTrove the approve amount is **net debt (debt – 20 HCHF)**, not the displayed full debt (§2.1). Exact amounts make residue zero by construction for the **fixed-amount flows** (`provideToSP`, `repayHCHF`, `stake`) — redemption is not fixed-amount: hints can truncate the consumed amount (§05 step 4d), so approve the truncated amount and check the allowance immediately after; zero residue is not absolutely guaranteed for large redemptions on the immutable contracts.
- **Residue allowance check + auto-zero.** Post-execution, query `/accounts/{user}/allowances/tokens`; residue must be 0. If any residue is found (truncated redemption, non-atomic fallback, partial failure), surface a one-click revoke and block further batched operations until cleared. For redemption this check is mandatory, not optional — it is the only way to catch truncation residue.
- **HashPack version gate.** Detect < 14.4.0 before connecting and show an upgrade path; do not silently degrade.
- **Graceful fallback = warn-and-proceed non-atomic.** For wallets without batch support: explicit warning that the 7-second exposed window returns, a recommendation to use the §05 CLI



instead, and — if the user proceeds — a mandatory post-tx revoke prompt. Fallback must never be silent.

- **Honest fee estimate.** Show max fee covering the batch fee + each inner's full fee, and state that inner fees are charged even if the batch rolls back (§4.2).

## 6.6 Security checklist

- Exact-amount approvals only; no unlimited approvals anywhere in the fork (including legacy non-batched paths).
- **closeTrove approvals are exactly net debt (debt – 20 HCHF), never the full debt** — approving the full debt leaves a standing 20 HCHF allowance to the wrapper, sweepable by the same mechanism this guide closes (§2.1). Unit-test the builder on this: approve amount must equal displayed debt minus 20 HCHF (GasPool compensation).
- batchKey is the user's own public key from the wallet session; the frontend never sees, holds, or transmits a private key.
- All contract/token ids from the 2026-10-05 fingerprinted set (§05 step 3) — never from repo configs; ship a verified address manifest with the frontend.
- Canonical HLQT HTS 0.0.6070128 only — counterfeit-HLQT confusion is an active risk (§08); verify wrapper 0.0.6070127 is the spender before building.
- Enforce the invariant in code: exactly one contract op, last position; unit-test the builder against `INVALID_TRANSACTION_BODY` shapes.
- Redemption path decodes and uses the third `getRedemptionHints` return (`truncatedHCHFAmount`): approve and call with the truncated amount, or abort — never approve more than the call consumes (§05 step 4d).
- Log batch id + inner ids for every operation (audit + incident replay); mirror-node residue check runs before showing success.
- Pinned dependency versions; WalletConnect project id locked to the deployer's; supply-chain review when bumping the connector to 2.1.x.
- Testnet sign-off before any mainnet enablement; mainnet pilot with small amounts first (§07).

## 6.7 HashPack test plan (6 scenarios)

| #  | SCENARIO                                                | PROCEDURE                                                                                                                                                                                                                                                            | PASS CRITERIA                                                                                                                                                                                                        |
|----|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T1 | Version gate                                            | Connect with HashPack 14.3.x, then with $\geq 14.4.0$                                                                                                                                                                                                                | 14.3.x: blocked with explicit upgrade message, no silent fallback. $\geq 14.4.0$ : proceeds.                                                                                                                         |
| T2 | Inner rendering at signing<br><b>PARTIALLY RESOLVED</b> | Mainnet validation (§3.3) already observed <b>three prompts</b> (inner 1, inner 2, outer batch). On testnet, build a 2-inner batch (associate + approve, then approve + redeem) and capture what HashPack displays <i>inside</i> each prompt.                        | Record whether inner details are shown per prompt or summarized — the remaining UX unknown; dapp-side preview matches wallet content either way.                                                                     |
| T3 | Happy path (testnet)                                    | Batched redeemCollateral end-to-end via the <b>validated per-transaction path</b> (§6.4: sign inner 1, inner 2, outer batch on hedera-wallet-connect 1.5.1; relay via <code>hedera_executeTransaction</code> ), incl. hint queries and the step-4d truncation check. | Three prompts; relayed batch executes; both inners SUCCESS; mirror node shows ATOMICBATCH parent + 2 inners sharing it; consumed amount = approved amount; residue allowance = 0 (checked immediately after commit). |
| T4 | Atomic rollback                                         | Batch a redeem that must revert (amount > balance)                                                                                                                                                                                                                   | Whole batch rolls back, <code>INNER_TRANSACTION_FAILED</code> , residue allowance = 0, both inner fees charged — and the fee estimate shown pre-tx covered reality.                                                  |
| T5 | User rejection                                          | Reject at the first of the three wallet prompts                                                                                                                                                                                                                      | Nothing submits; no allowance exists afterward; clean error state, retry offered.                                                                                                                                    |
| T6 | Non-atomic fallback                                     | Simulate a wallet without batch support                                                                                                                                                                                                                              | Warn-and-proceed with explicit 7-second-window warning + CLI recommendation; post-tx revoke prompt mandatory; success screen never appears without the residue check.                                                |

## 07 Proposed community timeline — 20 weeks to the sunset

Week 1 begins 2026-10-05. This proposed community timeline treats “March 2027, estimated” as a hard stop, holds a user-exit completion buffer of **2027-01-31** (end of week 17), and keeps February 2027 free of any new dependency on contract-call inners. If the sunset slips, the buffer absorbs it; if it arrives early, only the router protects redemptions — and trove operations must already be

finished, because a router cannot perform them for existing users (§3.2). Nothing here is mandated — contributors adopt dates by picking up the work. **INFERENCE** (date handling) /

**RECOMMENDATION** (plan)

Track (week 1 = 2026-10-05)

W1W3W5W7W9W11W13W15W17W19W21 2027-01-31 · batched exits complete

**Batch exit track** — testnet validation (CLI + wallet)

Frontend build — BatchTxBuilder + 7 flows

Community beta — testnet, then small mainnet pilot

Mainnet rollout + user comms (incl. revoke campaign)

Buffer — mop-up, exceptions, non-HashPack users

**Router track (parallel)** — redemption-only router: build → audit → deploy

V2 decision & design — community migration analysis (parallel)

### Milestones

≈ Mar 2027 (est.) · contract-call inners removed — HARD STOP

batch-exit track router track (redemption only) v2 track (dashed) 2027-01-31 buffer milestone hard stop (estimated)

Dependency rules that make this plan safe: (1) weeks 1–8 validate everything on testnet — no mainnet enablement before scenario T1–T6 pass; (2) by **2027-01-31** every user who needs a batched exit has completed it — including every closeTrove at the corrected net-debt approval amount (§2.1); (3) from February 2027 nothing new depends on contract-call inners — the buffer exists precisely because “March 2027” is an estimate, not a committed date; (4) the redemption-only router goes live inside the batch window (weeks 15–16) so redemptions cross from bridge to durable path before the stop; trove operations cannot be routed (§3.2) and must complete inside the batch window or await migration/v2; (5) user comms run continuously through rollout — allowance-revocation guidance stays live throughout (member-facing summary in §3.4).

## 08 Risks & open questions

| RISK / OPEN QUESTION                                                 | DETAIL                                                                                                                                                                                                                                                                                                                                                      | MITIGATION / WHO PICKS IT UP                                                                                                     |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Sunset slips — or arrives early</b>                               | “March 2027, estimated” is not a committed date. Earlier removal would cut the bridge short with users still exposed; a later one changes nothing about the router’s value. <b>INFERENCE</b>                                                                                                                                                                | 2027-01-31 completion buffer; router live by weeks 15–16; watch Hedera comms weekly (frontend contributors).                     |
| <b>HashPack inner-rendering UX partially resolved</b>                | The community mainnet test observed <b>three signing prompts</b> (§3.3), but no public description exists of whether HashPack shows each inner’s details or a summary inside those prompts. <b>UNVERIFIED</b> (rendering) / <b>CONFIRMED</b> (prompt count)                                                                                                 | Scenario T2 in §6.7; dapp-side preview authoritative until resolved.                                                             |
| <b>Wallet batch flow — validated, but only on one connector line</b> | The assembled no-operator freeze + wallet-sign flow has no public example, but a community contributor reproduced it on mainnet (§3.3) with hedera-wallet-connect 1.5.1: three prompts, relay via <code>hedera_executeTransaction</code> . The same pattern on other connector versions is not yet evidenced. <b>CONFIRMED</b> (community-validated, 1.5.1) | Keep §6.7 T2–T3 as the regression gate; re-run on every connector version bump before mainnet use.                               |
| <b>Fee-on-rollback economics</b>                                     | Each inner pays full fees even when the batch rolls back; a failed redeem costs real fees and leaves the user unredeemed. <b>CONFIRMED</b>                                                                                                                                                                                                                  | Read-only pre-flight (price/hints/balance) before building; honest max-fee UI; gas table calibrated on testnet.                  |
| <b>Bot adapts / fallback-path exposure</b>                           | Batching closes the window only for batched flows on the upgraded frontend. Non-atomic fallbacks, other frontends, and other dapps granting wrapper allowances remain sweepable; the bot is still active (~30–60 s cycle — community monitoring). <b>CONFIRMED</b> (bot) / <b>INFERENCE</b> (adaptation)                                                    | Warned fallback + revoke prompt; keep monitoring and flagging (community IOC list + mirror-node watcher rules).                  |
| <b>Counterfeit HLQT</b>                                              | Token-confusion risk when users approve HLQT: only the canonical HTS 0.0.6070128 via wrapper 0.0.6070127 is legitimate. <b>CONFIRMED</b> (existence of counterfeits confirmed in the community analysis)                                                                                                                                                    | Address manifest + id pinning in the builder; UI shows verified ids.                                                             |
| <b>Repo/mainnet config mismatch</b>                                  | Live mainnet deployment was never committed; committed <code>hedera.json</code> is testnet-era and mislabeled. Building from repo configs targets the wrong contracts. <b>CONFIRMED</b>                                                                                                                                                                     | Fingerprinted set only (§05 step 3); publish the verified manifest; fix or clearly tombstone repo configs. <b>RECOMMENDATION</b> |

| RISK / OPEN QUESTION                          | DETAIL                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | MITIGATION / WHO PICKS IT UP                                                                                                                     |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Router scope (narrowed, community-2.0)</b> | <p>Durable router = <b>permissionless redemption only</b>: closeTrove/adjustTrove act on <code>msg.sender</code>, so a router would be the caller, not the trove owner — it cannot perform those operations for existing users. SP exits, unstaking, claims, and claimCollateral need <b>no router</b> (no allowance involved; standalone calls keep working after March 2027). Existing-user trove operations = batches until cutoff / migration / v2.</p> <p><b>RECOMMENDATION</b></p> | <p>Scope the router to redemption (§3.2); keep trove-exit completion inside the 2027-01-31 buffer (§07); migration/v2 track covers the rest.</p> |
| <b>Connector version</b>                      | <p>Fork pins hedera-wallet-connect ^1.4.2; the validated signing path ran on 1.5.1 (satisfies ^1.4.2); latest is 2.1.x. <b>CONFIRMED</b></p>                                                                                                                                                                                                                                                                                                                                             | <p>Decide upgrade vs. pin during weeks 3–6; either way: no hashconnect, and re-run §6.7 T2–T3 on the new version before mainnet.</p>             |
| <b>#694 as a version caveat</b>               | <p><code>DAppSigner.signTransaction</code> rebuilds the <code>ContractExecute</code> body on some connector versions, breaking naive sign-then-submit. The validated 1.5.1 per-transaction path is unaffected; the caveat binds on version bumps, not today.</p> <p><b>CONFIRMED</b> (issue open) / <b>INFERENCE</b> (version scoping)</p>                                                                                                                                               | <p>Re-test the §6.4 signing path at each connector bump; until a version fails, the validated per-transaction pattern stands.</p>                |

## 09 Evidence & verification appendix

### 9.1 Primary sources (all API claims verified against these in this research round)

| CLAIM AREA                                                                                                                                                                        | SOURCE                                                                                                                                                            | PATH / URL                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HIP-551 spec (Final; release v0.66.0; mainnet re-enabled 2025-10-09)                                                                                                              | Hedera Improvement Proposal                                                                                                                                       | <a href="https://hips.hedera.com/hip/hip-551">https://hips.hedera.com/hip/hip-551</a>                                                                                                                   |
| One-contract-op-last rule; forbidden inner types; fees-on-rollback; batchKey signing                                                                                              | Hedera documentation "Create a Batch Transaction" + consensus-node source<br>( <code>AtomicBatchHandler.pureChecks</code> ,<br>hier-ledger/hedera-consensus-node) | <a href="https://docs.hedera.com">docs.hedera.com</a> ·<br><a href="https://github.com/hiero-ledger/hedera-consensus-node">github.com/hiero-ledger/hedera-consensus-node</a>                            |
| JS SDK<br>BatchTransaction API<br>( <code>addInnerTransaction</code> ,<br><code>batchify</code> , property<br>getters; ≥2.64.0; also<br><code>@hiero-ledger/sdk</code><br>2.89.x) | SDK JSDoc + repo examples                                                                                                                                         | npm <code>@hashgraph/sdk</code> · SDK repo<br><code>examples/batch-tx.js</code> (batchify<br>+ multi-batchKey patterns)                                                                                 |
| Complete official batch example                                                                                                                                                   | Hedera tutorial repo                                                                                                                                              | <a href="https://github.com/hedera-dev/tutorial-js-hip-551-batch-transaction">github.com/hedera-dev/tutorial-js-hip-551-batch-transaction</a><br>( <code>mintFractionalAsset.js</code> )                |
| March 2027<br>deprecation (quotes in §3.1)                                                                                                                                        | Official Hedera blog, 2026-09-22 —<br>"Atomic Batch Transactions No Longer<br>Support Smart Contract Calls"                                                       | <a href="https://hedera.com/blog">hedera.com/blog</a> (2026-09-22)                                                                                                                                      |
| 2023 precedent<br>(emergency shutoff,<br>v0.34.5, HSCS Security<br>Model v2)                                                                                                      | Official Hedera post-mortem, 2023-<br>03-11                                                                                                                       | <a href="https://hedera.com/blog/analysis-remediation-of-the-precompile-attack-on-the-hedera-network/">https://hedera.com/blog/analysis-remediation-of-the-precompile-attack-on-the-hedera-network/</a> |
| hashconnect<br>deprecated / shut<br>down; repo 404;<br>HashPack 14.3.0<br>removed legacy<br>protocol                                                                              | npm registry + GitHub + HashPack<br>patch notes                                                                                                                   | npm <code>hashconnect</code> (final 3.0.14,<br>2025-10-02) · HashPack patch<br>notes 14.3.0 (2026-06-24)                                                                                                |
| HashPack ≥14.4.0<br>batch support; 14.4.1<br>dapp-auto-approval;<br>auto-associate removal                                                                                        | HashPack patch notes                                                                                                                                              | HashPack 14.4.0 (2026-07-17),<br>14.4.1                                                                                                                                                                 |

| CLAIM AREA                                                                                                              | SOURCE                                                                                                                                        | PATH / URL                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| HIP-820 connector + methods; #694 bug                                                                                   | <code>@hashgraph/hedera-wallet-connect</code> (2.1.x in 2026) + issue tracker                                                                 | npm/github <code>@hashgraph/hedera-wallet-connect</code> · issue #694                                             |
| blokk fork stack (hedera-wallet-connect ^1.4.2 + sdk 2.72.0)                                                            | Repo inspection                                                                                                                               | github.com/blokk-studio/HLiquidity · github.com/SwisscoastAG/HLiquidity                                           |
| Reference implementation (CLI + frontend TypeScript)                                                                    | Community reference code                                                                                                                      | published alongside this document in the repository under <code>code/</code> — github.com/SwisscoastAG/HLiquidity |
| Mainnet contract set (fingerprinted 2026-10-05)                                                                         | On-chain fingerprinting vs. both repos                                                                                                        | ids as listed in §2/§5 of this document                                                                           |
| Community mainnet validation of the batched redemption — reproduction facts, three signing prompts, zero residue (§3.3) | Community contributor mainnet test (contributor unnamed)                                                                                      | the GitHub issue in github.com/SwisscoastAG/HLiquidity                                                            |
| closeTrove net-debt burn and 8-decimal maxFeePercentage (community-2.0 corrections)                                     | Repo-verified against the deployed contract source (BaseMath.sol DECIMAL_PRECISION = 1e8; BorrowerOperations.closeTrove GasPool compensation) | github.com/SwisscoastAG/HLiquidity · github.com/blokk-studio/HLiquidity                                           |
| Incident campaign figures (victims, waves, \$ ledger, open allowances)                                                  | Community forensic analysis of mainnet data (methodology in §2/appendix)                                                                      | mirror-node REST + HashScan, endpoints in §9.2                                                                    |

## 9.2 Measurement endpoints

base: <https://mainnet-public.mirrornode.hedera.com/api/v1> # testnet:

<https://testnet.mirrornode.hedera.com/api/v1> # batch verification: parent (type ATOMICBATCH) + inners (batch\_key, parent\_consensus\_timestamp, nonce) GET /transactions/{batchId} # per-owner allowance state — residue must be 0 after a committed batch GET /accounts/{accountId}/allowances/tokens # reference incident tx (the 1,800 HCHF drain, 2026-09-29) GET /transactions/0.0.995584-1790680920-326001014 # same transaction, browser view <https://hashscan.io/mainnet/transaction/0.0.995584-1790680920-326001014>



### 9.3 Verification status of load-bearing claims

- **CONFIRMED** — HIP-551 semantics and limits; the one-contract-op-last rule; fee-on-rollback; batchKey signing model; the deprecation blog and its exact quotes; hashconnect deprecation; HashPack  $\geq 14.4.0$  batch support; issue #694; blokk fork stack; mainnet contract set; all incident/\$ figures from community forensic analysis of public mainnet data. **Community-validated on mainnet (§3.3)**: the assembled hedera-wallet-connect + BatchTransaction flow; the no-operator freeze + wallet-signing pattern; HashPack  $\geq 14.4.0$  batch signing in practice via connector 1.5.1 (three signing prompts observed; signed bytes relayed via `hedera_executeTransaction`; exactly 100,000,000 units consumed; zero residue allowance).
- **RECOMMENDATION** — batch-every-flow policy; router design (permissionless-redemption scope — §3.2/§08); v2 endgame per the community migration analysis (summarized in §3.2); UX, security, and test-plan specifications; timeline structure.
- **INFERENCE** — race-elimination claim (no window exists by construction; residual exposure limited to fallback paths, non-upgraded surfaces, and possible truncation residue on large redemptions — §05 step 4d); treating “March 2027, estimated” as a hard planning date; bot-adaptation outlook.
- **UNVERIFIED** — how HashPack renders the inner details inside each signing prompt; the single-prompt `hedera_signAndExecuteTransaction` path for a batch containing a contract call.

### 9.4 Changelog

**community-2.0 (2026-10-05)**. Mainnet validation by a community contributor; corrections to closeTrove approval amount, maxFeePercentage decimals, redemption truncation, router scope, and signing patterns. One line per correction:

- **Mainnet validation — PROVEN (§3.3)**: a community contributor reproduced the batched redemption on mainnet — `[CryptoApproveAllowance 1 HCHF] + [TroveManager.redeemCollateral]` as the final inner under one parent AtomicBatch; the batch succeeded, exactly 100,000,000 units were consumed and burned, HBAR was returned, and the residue allowance was zero; the mirror record confirms ordering and shared parent. Evidence in the GitHub issue; the contributor remains unnamed unless they request attribution. Three signing prompts observed (inner 1, inner 2, outer batch).
- **closeTrove approval amount (safety-critical, §2.1/§05/§6.3/§6.6)**: approve exactly net debt (debt – 20 HCHF); the GasPool provides the 20 HCHF gas compensation. Approving the full debt leaves a 20 HCHF stealable allowance.
- **maxFeePercentage decimals (§05)**: corrected to the deployment’s 8-decimal fixed point (`BaseMath DECIMAL_PRECISION = 1e8`):  $100\% = 1e8$ ,  $0.5\% \text{ floor} = 500\_000$ .
- **Redemption truncation (§05 steps 4d–5, §6.5)**: decode and use the third `getRedemptionHints` return ( `truncatedHCHFAmount` ) — approve and call with the truncated amount, or abort; never approve more than the call consumes. Stale hints or state changes can still cause partial consumption: check the allowance immediately after. No absolute zero-residue guarantee for large redemptions on the immutable contracts; the zero-residue-by-construction phrasing now applies to fixed-amount flows only (`provideToSP`, `repayHCHF`, `stake`).

- **Router scope narrowed (§3.2/§07/§08):** durable router = permissionless redemption ONLY — closeTrove/adjustTrove act on `msg.sender` and cannot be routed for existing users; SP exits, unstaking, and claims need no router (standalone calls keep working after March 2027); existing-user trove operations = batches until cutoff / migration / v2.
- **Signing patterns (§6.4/§6.5/§6.7):** per-transaction signing (three HashPack prompts via hedera-wallet-connect 1.5.1, relay via `hedera_executeTransaction` ) validated on mainnet; the single-prompt `hedera_signAndExecuteTransaction` path remains unvalidated; issue #694 reframed as a version-dependent caveat, not a mandate.
- **Test defaults labeled (§05):** fixed gas values and the zero-hints fallback are test defaults — calibrate on testnet, use live hints from HintHelpers, and cap maxIterations deliberately.

community-1.0 (2026-10-05): initial public community edition (successor to the internal engineering guide; referred to in this repository as the previous community edition). community-2.0 is the current edition; corrections always via GitHub issues.

**Disclaimer.** This is an independent, community-produced engineering analysis based solely on public Hedera mainnet data (mirror-node REST API, HashScan), official Hedera sources (HIP-551, documentation, consensus-node source, the 2026-09-22 deprecation blog, the 2023 post-mortem), npm registry and wallet patch notes, and the cited public GitHub repositories. No entity operates the HLiquity protocol; this document was produced by community contributors from public mainnet data. It is not affiliated with, or endorsed by, Swisscoast AG, HashPack, Hedera, or any audited protocol. Address references describe on-chain accounts; they are not accusations against identified natural persons or legal entities. Not financial advice. Corrections via GitHub issues only.